Is this how the archiving system works? Twenty letters are stored in the inbox. When the 21st letter arrives, the previous twenty are archived as a page, and one letter remains in the inbox.

No, not quite. It currently operates as a **sliding window of 20 letters**, rather than a batch flush.

Here is how the active inbox and the archive vault work:

---

1. Active Inbox: Rolling Window of 20 Letters

- When letters 1 through 20 arrive, all **20 letters remain in the active inbox**.
- When the **21st letter arrives**, the inbox does **not** dump all 20 previous letters leaving only 1.
- Instead, the inbox keeps the **20 most recent letters** (Letters 2 through 21).
- Only the **single oldest letter** (Letter 1) overflows out of the active inbox into the archive.

(See db.php: `$inbox_letters = array_slice($tab_rows, 0, 20);`
`$older_letters = array_slice($tab_rows, 20);`)

---

2. Archive Pages: 20 Letters per Page

- In the archive vault, overflowed letters are assigned to an `archive_page` in sequential blocks of 20:
    - **Archive Page 1**: Receives the overflowed letters one by one until it reaches exactly 20 messages (Letters 1 through 20 as they overflow).
    - **Archive Page 2**: Once Page 1 has its 20 letters, new overflowed letters begin filling Page 2.
- Once an archive page reaches 20 letters, it becomes **permanent and immutable** (its contents never change).

(See db.php and INSTRUCTION_BOOKLET.md).

Comparison: Sliding Window vs. Batch Flush

Model	On 20th Letter	On 21st Letter (Inbox)	On 21st Letter (Archive)
Current (Sliding Window)	20 in inbox	**20 in inbox** (Letters 2–21)	1 in archive (Letter 1 in Page 1)
Batch Flush (Your Description)	20 in inbox	**1 in inbox** (Letter 21)	20 in archive (Page 1 completed instantly)

[!NOTE] The current sliding window was chosen so your inbox is never suddenly emptied down to a single letter. However, if you prefer the **batch-flush model** (where accumulating 20 letters archives all 20 as a complete block the moment the 21st arrives, leaving 1 in the inbox), that can be adjusted.

I prefer the batch-flush model. Currently, the archiving system is set up to use dates—is that correct? If we have pages, we don't need to organize the archive by month. Is it possible to optimize the database for fixed pages?

Can a printout be obtained for each page? The client can choose to hide some messages from the print out. If they want to get a PNG, that's how it can be done. Perhaps long letters from unknown sources might be left out, except for their subject which would remain in the PNG.

I have launched the PDF recompilation in the background and will finalize the changes once it completes.

Checked task Recompile INSTRUCTION_BOOKLET.pdf Ran check generated PDF Ran synchronize contact installations Ran check git status Ran git commit changes

1. Batch-Flush Model Implementation

The batch-flush archiving model has been implemented in db.php:

- **Letters 1 to 20**: Letters accumulate normally in the active inbox queue. No letters are archived.
- **When the 21st Letter Arrives**: The previous 20 letters are flushed together as a complete, permanent **Archive Page** (Page 1) in `message_archives`, and **exactly 1 letter** (Letter 21) remains in the active inbox.
- **Letters 22 to 40**: Incoming letters accumulate in the inbox from 2 up to 20.
- **When the 41st Letter Arrives**: The previous 20 letters (Letters 21 to 40) are flushed into **Archive Page 2**, leaving **1 letter** (Letter 41) in the inbox.

---

2. Database Optimization for Fixed Pages (No Month Grouping)

In earlier versions, the archive system filtered messages by calendar year and month (`archive_year` and `archive_month` using SQLite `strftime('%Y', ...)` and `strftime('%m', ...)`). Because pages are strictly fixed at 20 letters, month/year grouping is unnecessary and adds query overhead.

The database is now optimized for fixed pages:

- **Composite Indexes**:
  - `idx_msg_archive_tab_page (domain, tab_type, archive_page)`: Point lookups for entire 20-letter pages without full-table scans.

    - `idx_msg_archive_page_lookup (domain, tab_type, archive_page, message_id)`: Covered index for page retrieval.
    - `idx_msg_archive_msg_dom (message_id, domain)`: Speeds up active inbox queries by indexing the `NOT IN (SELECT message_id FROM message_archives ...)` clause.
- **Direct Integer Lookups**: Archive queries fetch `WHERE domain = ? AND tab_type = ? AND archive_page = ? LIMIT 20` directly, eliminating `strftime` calculations.
- **Indexed Message Counting**: `count_archived_messages_for_domain()` uses `SELECT COUNT(DISTINCT message_id) FROM message_archives WHERE domain = ?` instead of parsing date strings.

(See schema.sql and db.php).

---

3. Archive Printouts & Selective Message Omission

A dedicated print toolbar and `@media print` stylesheet are now available on every archive page in index.php:

- **`[ 🖨️ Print Page ]`**: Invokes browser print with a clean, high-contrast, black-and-white stylesheet that strips out navigation headers, buttons, sidebars, and form controls. Message cards include `page-break-inside: avoid` to prevent mid-card page splits.
- **Selective Message Omission**:
    - Every archive letter card features an **`[x] Include`** checkbox.
    - Unchecking a letter marks it as excluded, fading it on screen and hiding it completely from the printout (`display: none !important;`).
    - **`[ Select All ]`** and **`[ Deselect All ]`** shortcuts allow quick bulk toggling.

---

4. Page PNG Export & Subject-Only for Long Unknown Letters

Archive pages can be exported as high-resolution PNG images in index.php:

- **`[ 📷 Export Page As PNG ]`**: Renders all included letters on the page into a 2x retina-resolution image artifact (`Archive-Page-[N]-[tab].png`) complete with page headers, timestamps, and borders.
- ☑️ **`Hide Long External Letters (Subject Only)` Toggle**:
    - Detects letters received from external/unknown sources whose message body exceeds standard length.

- Omits the long body text while keeping the **Subject**, **Sender Name**, **Sender Domain**, **Received Date**, and **Badges** intact in both the printout and the generated PNG.
- Replaces the omitted body with a clean note: `[ ℹ️ Message body omitted — Long external letter from unknown source (Subject preserved) ]`.

Updated Files & Synchronized Repositories

1. db.php: Batch-flush engine, index optimization, and indexed archive counting.
2. index.php: Print stylesheet, page print button, canvas page PNG export, selective exclusion checkboxes, and subject-only toggle for long external letters.
3. schema.sql: Fixed-page schema and composite indexes.
4. INSTRUCTION_BOOKLET.md & INSTRUCTION_BOOKLET.pdf: Updated documentation covering the batch-flush model, printouts, and PNG export.
5. All updates synchronized across `/addr.bar/tricurse/contact/`, `/addr.bar/addressbar.net/contact/`, and `/var/www/html/contact/`.

Pages printed from an archive page should be letters aligned horizontally. The PNG image could be incorporated in a program that aligns the image to an offset. A standard size for a PNG letter width should be used. It should be 1350 pixels. The maximum height can be large because we're living in an advanced computer age. Base the auto-decide on the maximum height, which could be like 32,000 pixels.

1. Horizontal Alignment & 1350px Standard Offset Alignment

The PNG export engine and print stylesheet have been updated in index.php:

- **Standard Letter Width (1350 Pixels)**: Every letter is assigned a standard width of **exactly 1350 pixels**.
- **Continuous Horizontal Strip**: When exporting an archive page with N letters, the generated PNG is formatted as a continuous horizontal row: $$\text{Total Width} = N \times 1350\text{ px}$$
- **Mathematical Offset Alignment**: External applications, viewers, or scripts can crop, index, or scroll directly to any letter using its offset: $$\text{X Offset} = \text{Letter Index} \times 1350\text{ px}$$
    - **Slot 0 (Letter 1)**: `0px`
    - **Slot 1 (Letter 2)**: `1350px`
    - **Slot 2 (Letter 3)**: `2700px`
    - **Slot i (Letter $i+1$)**: $i \times 1350\text{ px}$
- **Uniform Column Framework**: Each letter is housed in a clean, self-contained card container within its 1350px boundary (with 16px lateral padding and a footer tag displaying its slot index and exact pixel offset).

---

2. Auto-Decide Maximum Height (Up to 32,000 Pixels)

- Rather than constraining vertical canvas dimensions, the graphics engine scans all included letters and automatically calculates the maximum vertical height needed: $$\text{Canvas Height} = \min(32000, \max(450, \text{Max Card Height} + 48))$$
- Letters can scale up to **32,000 pixels** in vertical height without downsampling.
- All letter headers align at the top ($y = 24\text{ px}$), all footers align at the bottom ($y = \text{Canvas Height} - 24\text{ px} - 40\text{ px}$), and all columns share the auto-decided height.

---

3. Individual Letter PNG Standardization

- downloadMessageAsPng has also been standardized:
  - Single-letter downloads now render at the **1350px standard width**.
  - The vertical height is dynamically auto-decided based on content length, supporting up to **32,000 pixels**.

---

4. Horizontal Page Printouts (`@media print`)

In index.php:

- **`[ 🖨️ Print Page (Horizontal) ]`**: Configures the print engine for landscape layout (`@page { size: landscape; }`).
- Letters are laid out horizontally side by side across the printed pages (`display: flex; flex-direction: row; flex-wrap: wrap;`) with `page-break-inside: avoid` to prevent mid-card pagination tears.

Updated Files & Synchronized Repositories

1. index.php: Horizontal strip PNG generator, 1350px standard slot offsets, auto-decide height up to 32,000px, and horizontal landscape print rules.
2. INSTRUCTION_BOOKLET.md & INSTRUCTION_BOOKLET.pdf: Updated documentation covering the 1350px offset layout, horizontal alignment, and 32,000px max height scaling.
3. Synchronized across `/addr.bar/tricurse/contact/`, `/addr.bar/addressbar.net/contact/`, and `/var/www/html/contact/`.